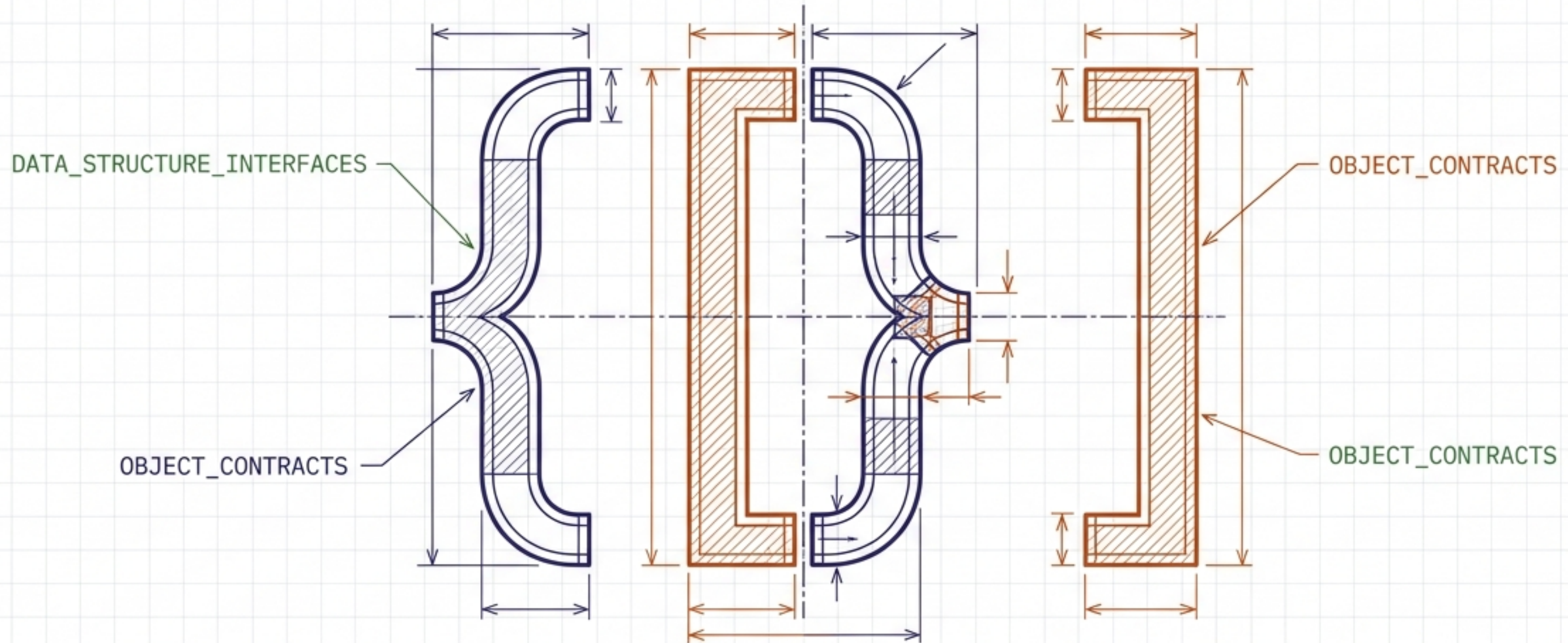


OCULTO SOB O CAPÔ: POLIMORFISMO E PROTOCOLOS EM PYTHON

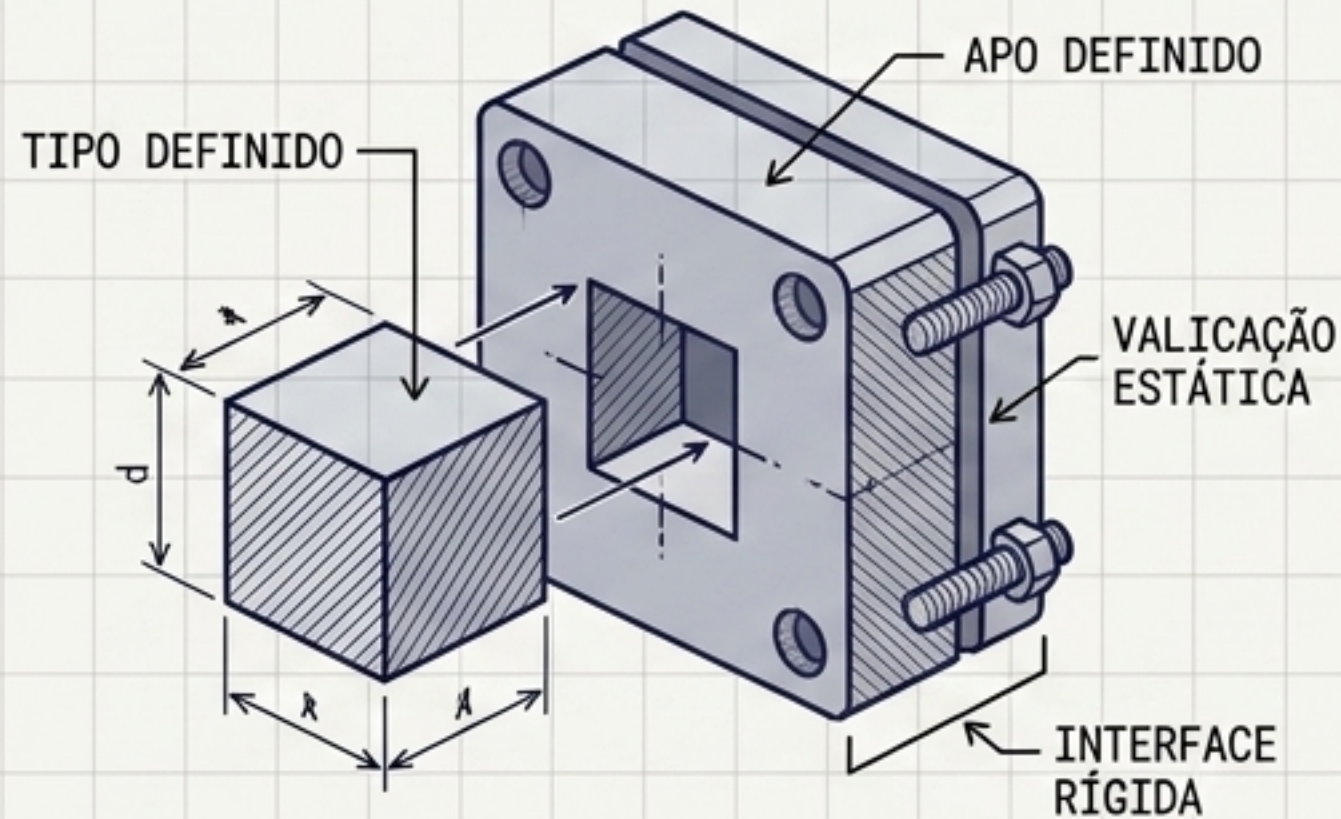
Uma radiografia técnica da flexibilidade orientada a objetos e dos contratos implícitos.



Sumário da Execução

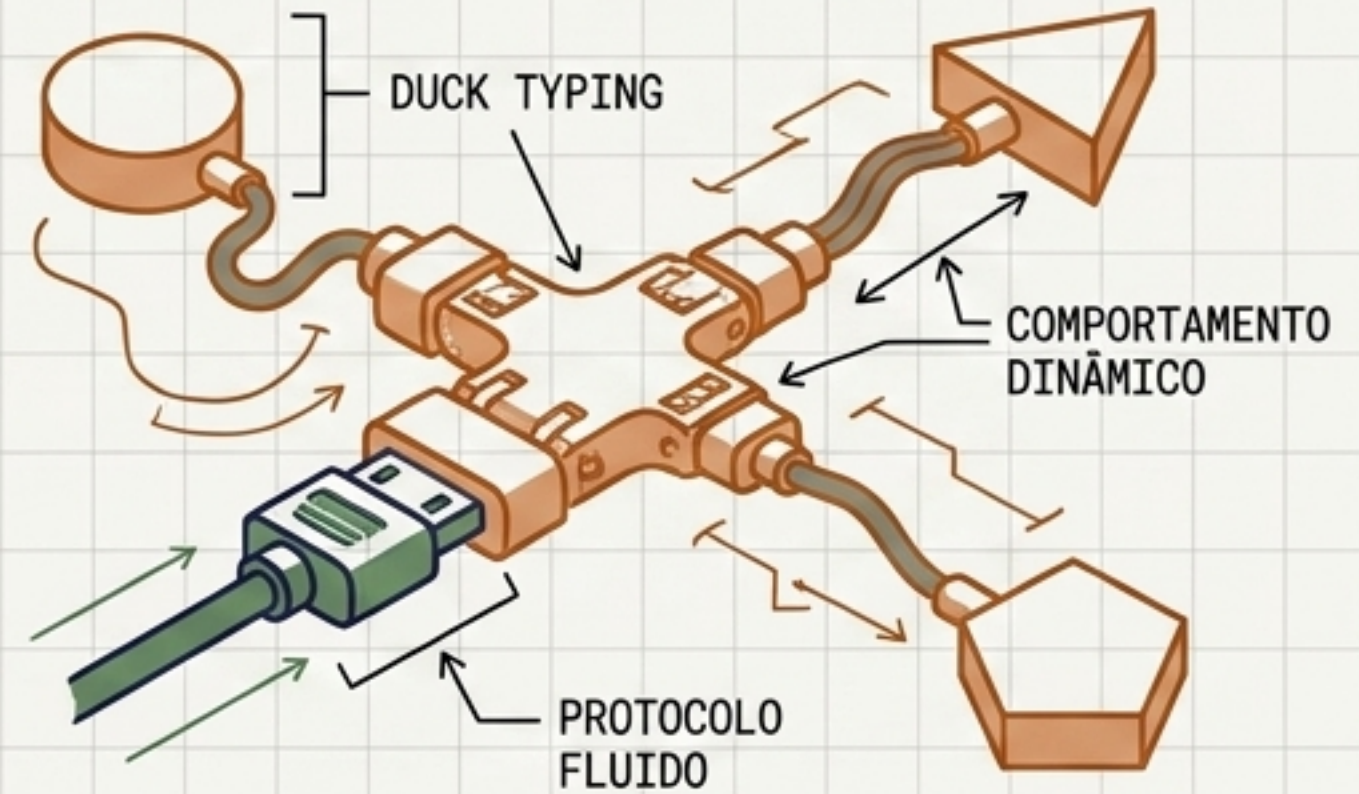
1. A Filosofia Central (Contratos e Duck Typing)	□
→ 2. Os Facilitadores (Protocolos e Polimorfismo)	○
→ 3. Mecânica Avançada 1 (Context Managers)	△
→ 4. Mecânica Avançada 2 (Descriptors)	◇
→ 5. Matriz de Síntese e Diagnóstico	⬠

Contratos Explícitos vs. Contratos Implícitos



Contrato Explícito (Java/C#)

Linguagens estáticas exigem garantias formais. Métodos e parâmetros devem declarar explicitamente seus tipos antes da execução.



Contrato Implícito (Python)

Python adota um fluxo livre. Não há o conceito estrito de uma 'interface'. A compatibilidade é verificada em tempo de execução com base no comportamento, não na linhagem.

A Filosofia do Duck Typing

Se anda como um pato, nada como um pato e grasna como um pato, então é um pato!

Em Python, a tipagem é estrutural (shape typing). Se um objeto pode executar o conjunto de operações exigido por um código, ele é perfeitamente válido para uso, independentemente de sua classe de origem.

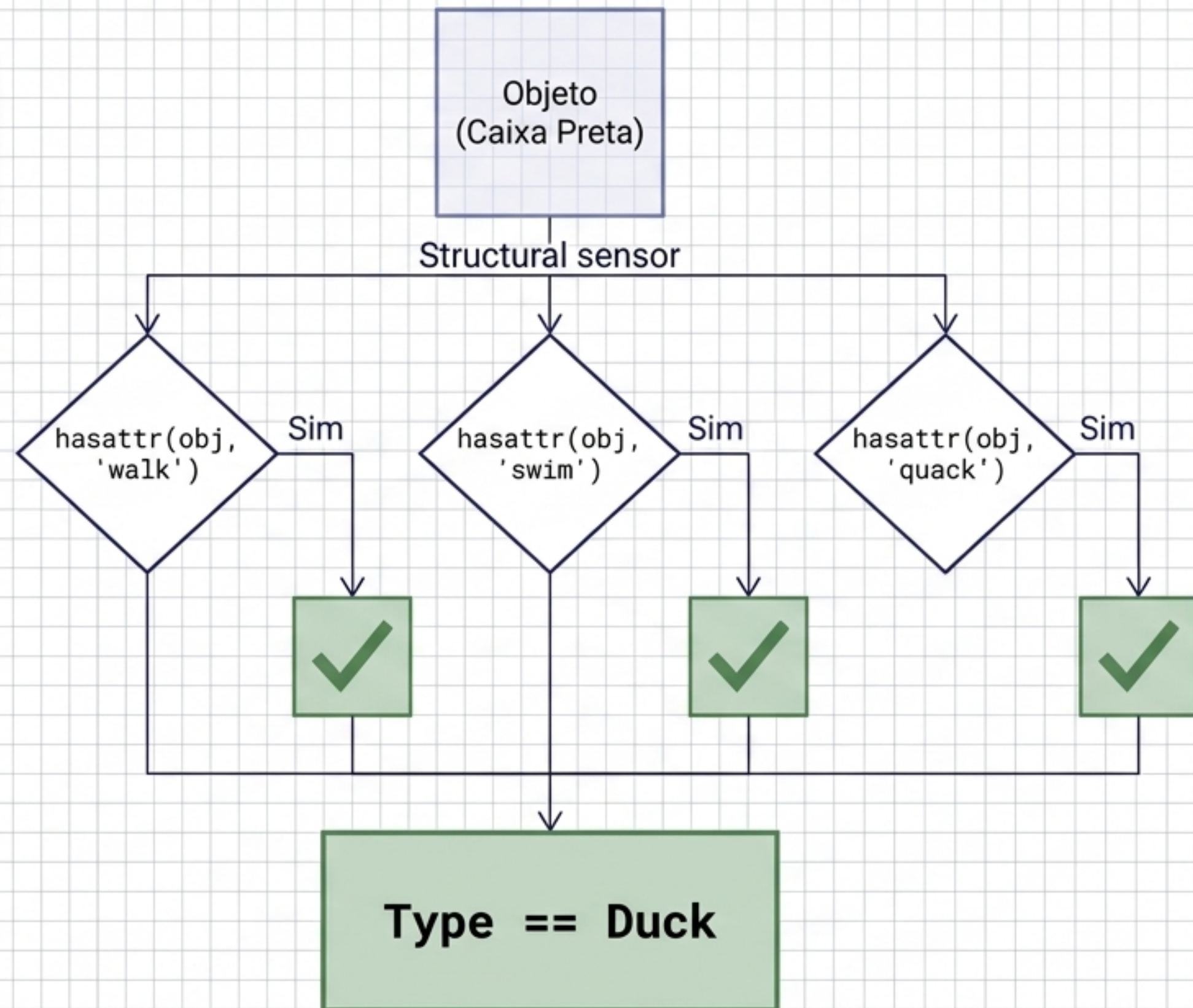


Ilustração Guiada: O Teste de Colisão

```
class Calculator:
    """ Simple Calculator class"""
    def add(self, x, y):
        return x + y

    def subtract(self, x, y):
        return x - y

    def multiply(self, x, y):
        return x * y

    def divide(self, x, y):
        return x / y
```

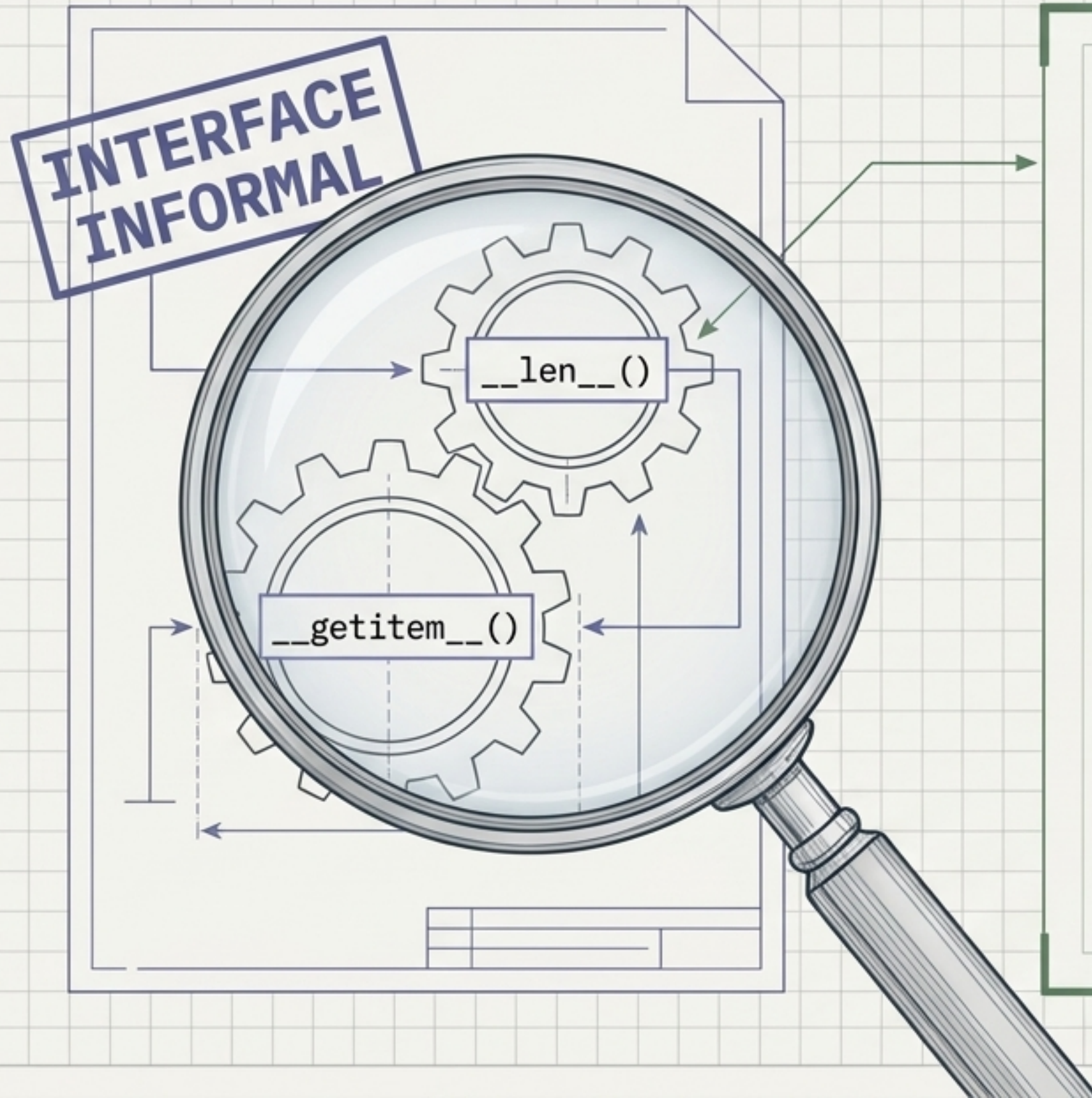
```
class Distance:
    def __init__(self, d):
        self.value = d
    def __add__(self, other):
        return Distance(self.value + other.value)
    def __sub__(self, other):
        return Distance(self.value - other.value)
    def __str__(self):
        return 'Distance[' + str(self.value) + ']'
```

Sucesso: Objetos da classe Distance funcionam perfeitamente no método add() da calculadora, pois o contrato parcial foi atendido.

Falha: A calculadora quebra no método divide(), pois a classe Distance não suporta o operador '/'. Você só precisa implementar o que efetivamente será executado.

TypeError: unsupported operand type(s) for /: 'Distance'

Protocolos: A Documentação como Lei

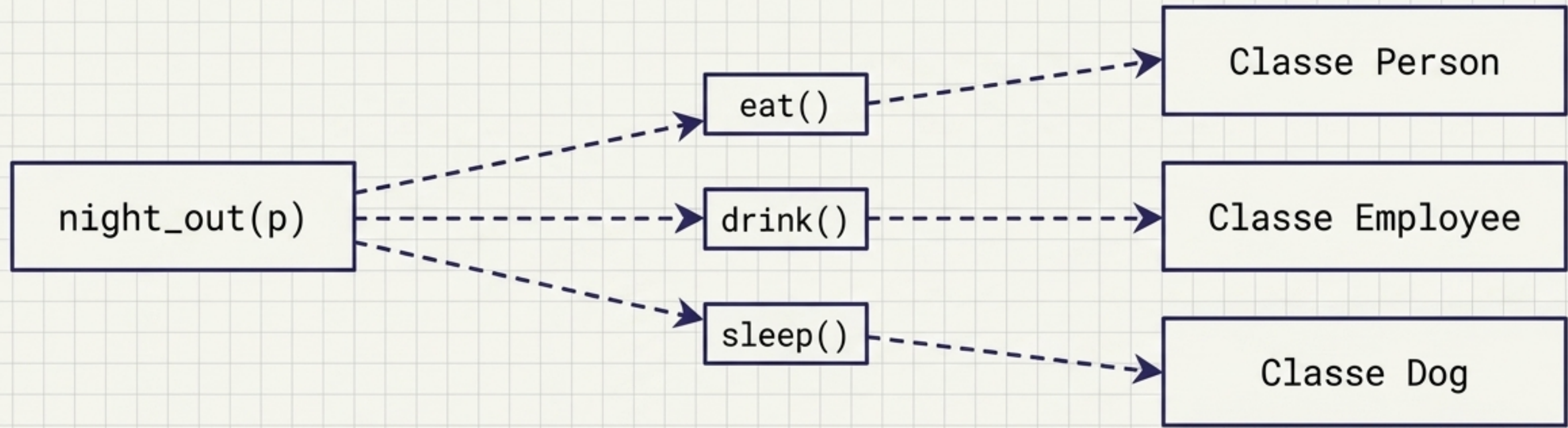


Como sabemos o que é exigido pelo Python? Através dos **Protocolos**.

Os **Protocolos** são descrições **informais** da **interface** do programador, definidas unicamente pela documentação.

O **Protocolo de Sequência**: Qualquer contêiner que deseja ser acessado item por item deve implementar os métodos mágicos exigidos. Nenhuma herança forçada é necessária; apenas a implementação exata da interface documentada.

O Protocolo Polimórfico



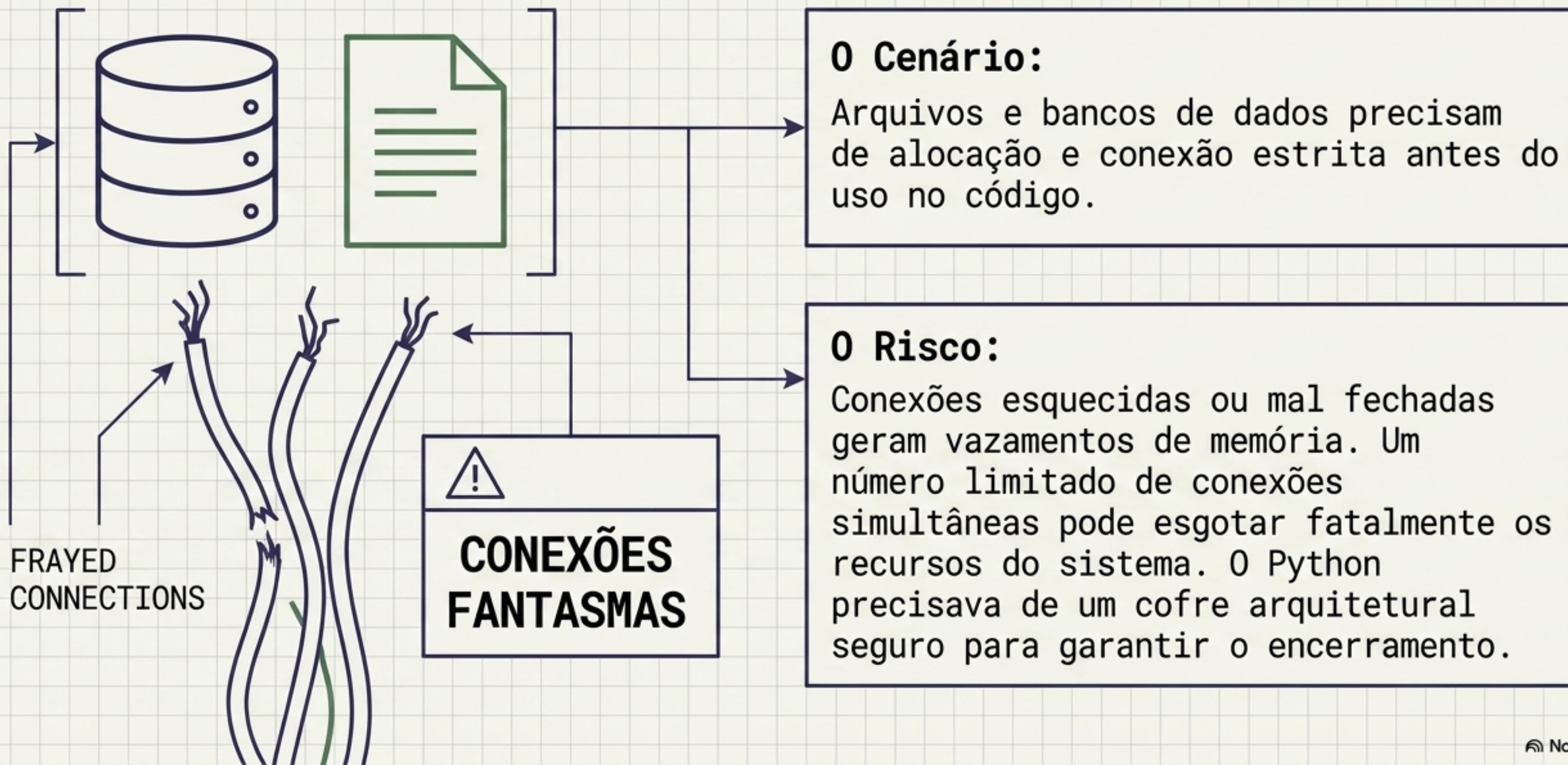
A Definição:

A habilidade de enviar a mesma mensagem (chamada de método) para objetos radicalmente diferentes.

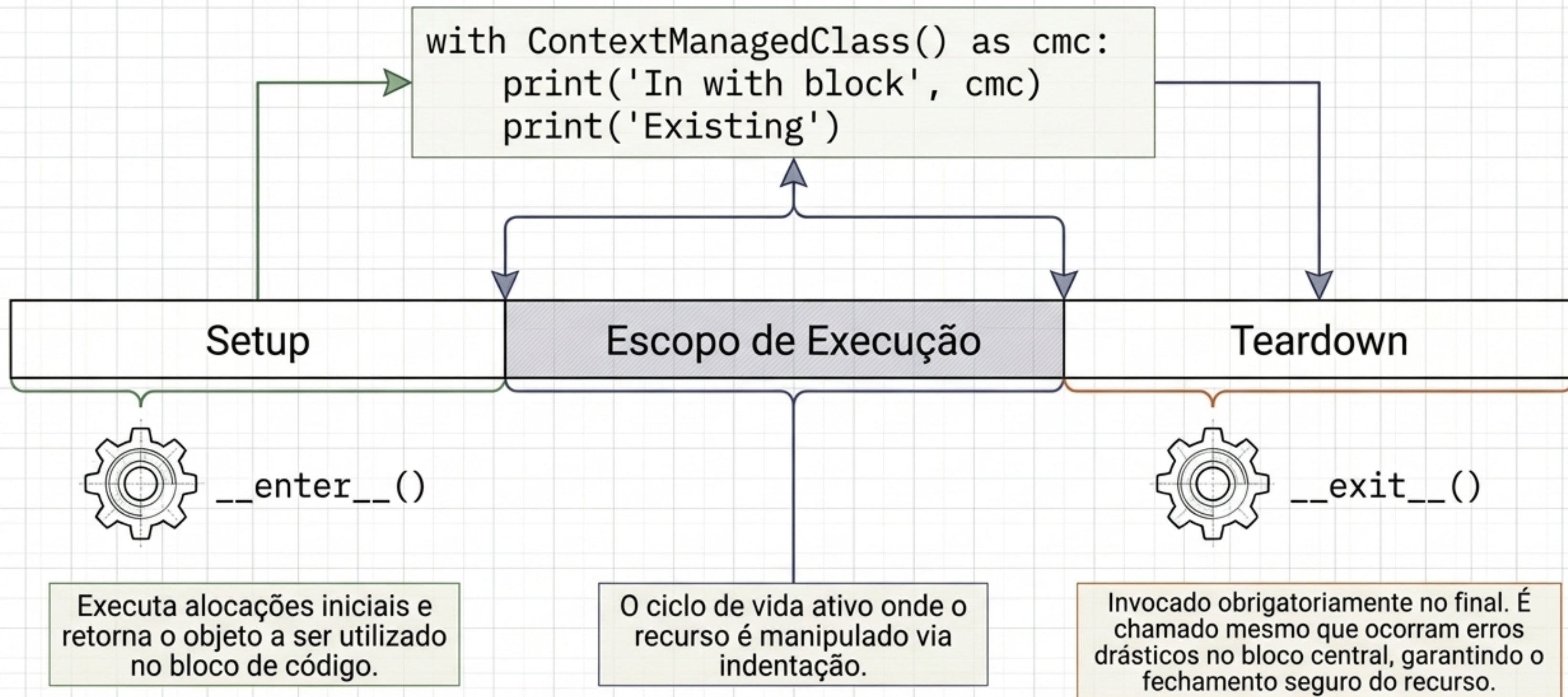
A Mecânica:

Como a mensagem é processada depende da classe receptora. O programador que envia a chamada não precisa se preocupar com a forma como ela será tratada, apenas garantir que o objeto atenda ao contrato informal.

Deep Dive 1: O Problema do Gerenciamento de Recursos



A Anatomia do Bloco with ... as

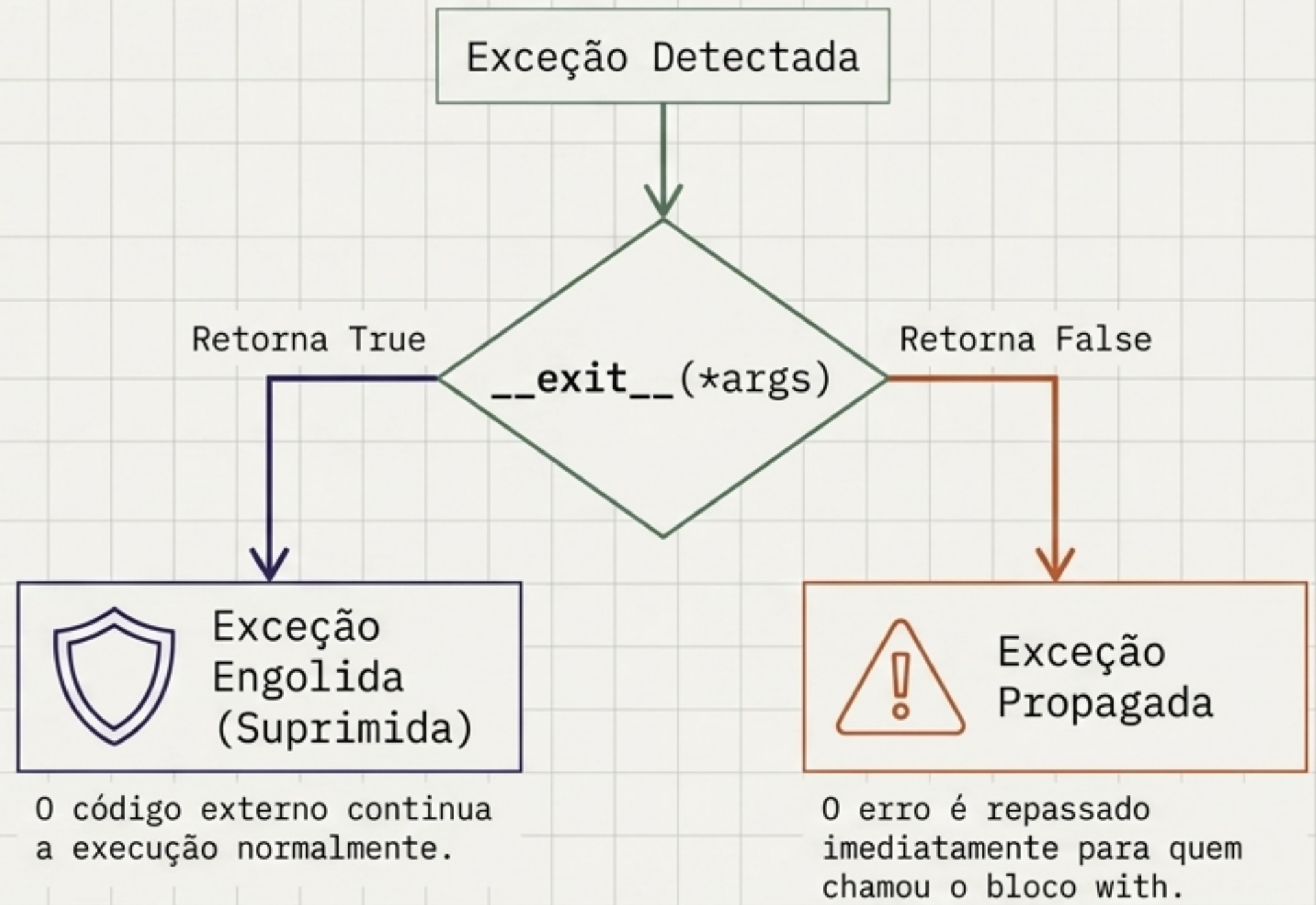


Controle de Danos: Tratamento de Exceções

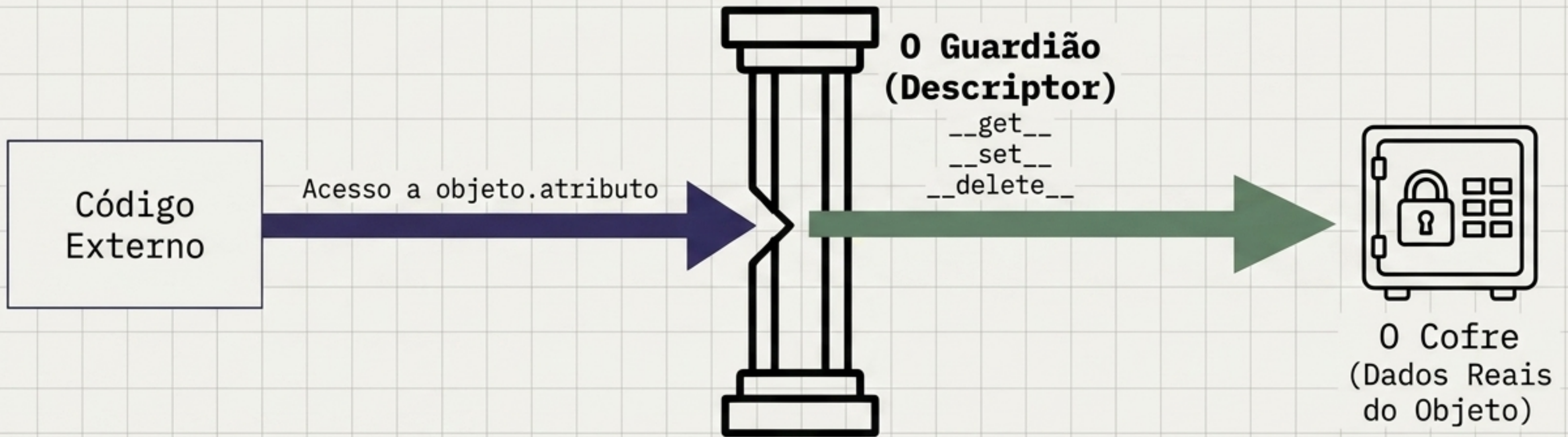
A Assinatura:

O método `__exit__` recebe automaticamente informações vitais sobre qualquer exceção gerada no corpo do bloco.

O comportamento pós-erro do sistema é ditado exclusivamente pelo valor booleano retornado.



Deep Dive 2: O Protocolo Descriptor



O Conceito

Criar atributos 'gerenciados'. Substitui o acesso direto ingênuo por uma camada de interceptação sofisticada.

O Propósito

Proteger e monitorar o acesso direto aos dados. O Descriptor pode validar formatos, auditar modificações ou atualizar campos relacionados de forma completamente invisível para a classe cliente.

Dissecando a Mecânica do Descriptor

Estes quatro métodos especiais transformam uma classe comum em um interceptador de nível de atributo.



`__get__(self, instance, owner)`

Invocado na leitura do atributo.
Retorna o valor calculado ou levanta erro.



`__set__(self, instance, value)`

Invocado na escrita. Intercepta e gerencia a atribuição de novos valores aos dados.



`__delete__(self, instance)`

Impede ou gerencia exclusões do atributo para evitar corrupção de estado.



`__set_name__(self, owner, name)`

(Adicionado no Python 3.6). Identifica e armazena o nome do atributo no exato momento da criação da classe.

Ilustração Guiada: A Armadilha da Recursão

Definição da Classe Cursor

```
class Cursor(object):  
    # Set up the descriptors at the class level  
    x = Logger('x')  
    y = Logger('y')  
  
    def __init__(self, x0, y0):  
        # Initialise the attributes  
        # Note use of __dict__ to avoid using self.x notation  
        # which would invoke the descriptor behaviour  
        self.__dict__['x'] = x0  
        self.__dict__['y'] = y0  
  
    def move_by(self, dx, dy):  
        print('move_by', dx, ',', dy)  
        self.x = self.x + dx  
        self.y = self.y + dy  
  
    def __str__(self):  
        return 'Point[' + str(self.__dict__['x']) + ', '  
            + str(self.__dict__['y']) + ']
```

__init__
(Método de Inicialização)

cursor.x = 10
(Gera Recursão)

~~Interceptor
(Erro)~~

~~cursor.x = 10
(Gera Recursão)~~



self.__dict__['x'] = x0
(Cofre de Dados Direto)

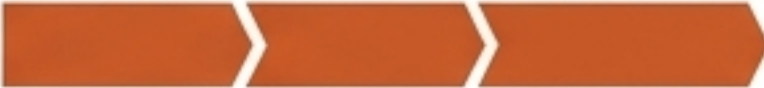
A Regra de Ouro:

Descriptors devem ser definidos no nível estrito da classe, jamais dentro da instância.

O Desvio Tático:

O método de inicialização do Cursor usa o dicionário interno `__dict__` para configurar os atributos iniciais. Isso contorna o mecanismo normal de busca (notação de ponto), evitando invocar o comportamento do interceptador prematuramente.

Matriz Arquitetural: Tipos de Contrato

Critério	Contratos Explícitos (Java/C#)	Duck Typing (Python)	Protocolos de Classe (Python)
Verificação de Tipo	Estrito e compilado	Dinâmico em tempo de execução	Documentado e orientado a métodos
Flexibilidade Visual Bar	 Baixa	 Extrema	 Alta e Estruturada
Enforcement	Garantido pelo compilador antes da execução	Informado por exceções (TypeErrors) apenas no momento da falha	Falha de contrato via comportamento faltante e documentação quebrada

Build Complete: Resumo Técnico

✓	1. Shape Typing	O Python prioriza o que um objeto pode fazer (comportamento em tempo de execução) sobre de onde ele veio (herança estrita).
✓	2. Protocolos Informais	Interfaces em Python são acordos baseados em documentação; suporte os métodos mágicos exigidos e o polimorfismo acontece de forma natural.
✓	3. Context Managers	O bloco "with" encapsula operações de risco estrutural, garantindo o gerenciamento perfeito do ciclo de vida de recursos através de <code>__enter__</code> e <code>__exit__</code> .
✓	4. Descriptors	Atuam como guardiões silenciosos entre o código externo e o dicionário de atributos interno do objeto, permitindo a criação poderosa de atributos autogerenciáveis.